



git

Day 4 – Software Carpentries

Alexander Garzón

Code of conduct

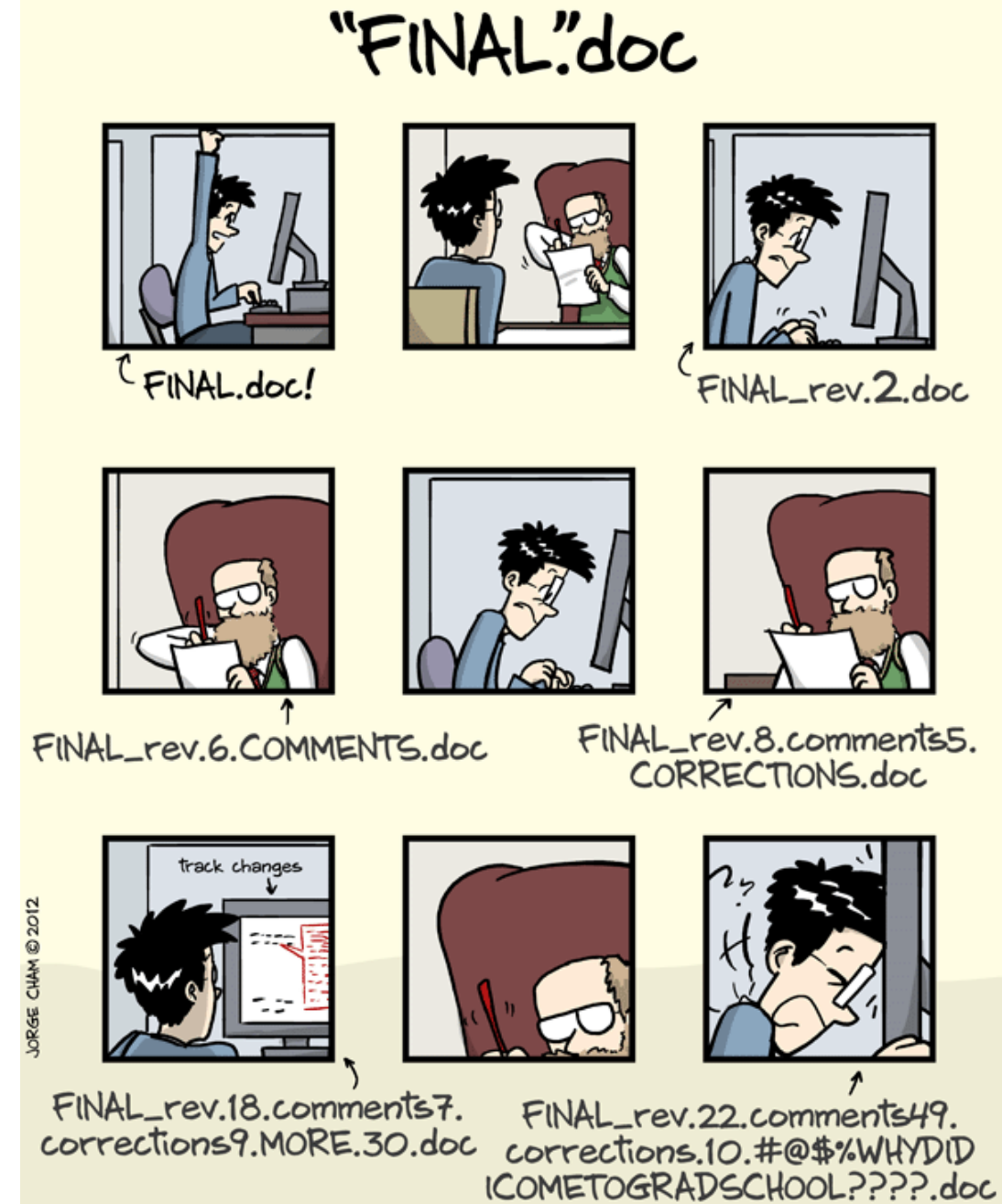
In order to foster a positive and professional learning environment we encourage the following kinds of behaviours in all platforms and events:

- Use welcoming and inclusive language
- Be respectful of different viewpoints and experiences
- Gracefully accept constructive criticism
- Focus on what is best for the community
- Show courtesy and respect towards other community members

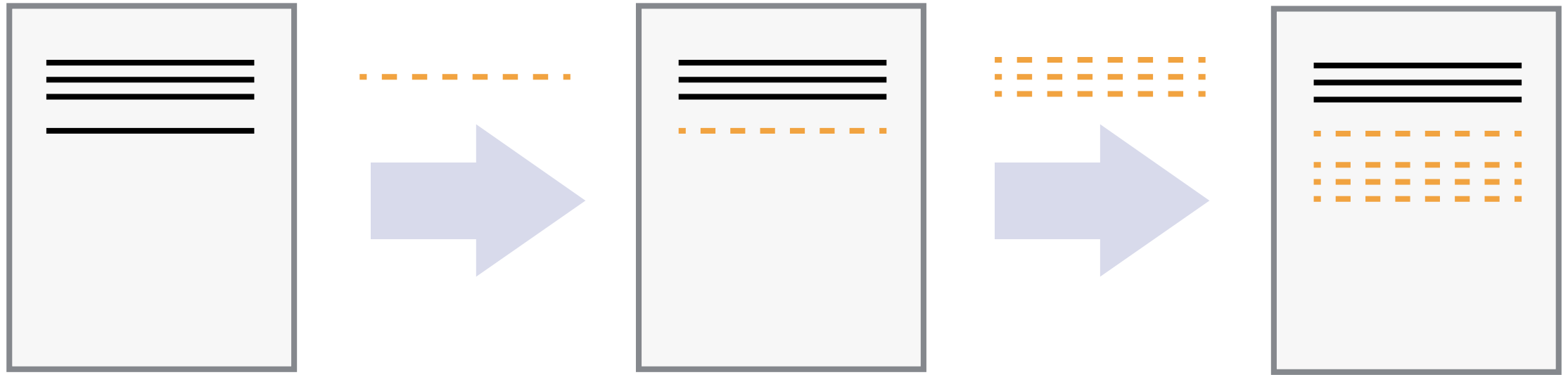
If you believe someone is violating the Code of Conduct, we ask that you report it.

Why version control?

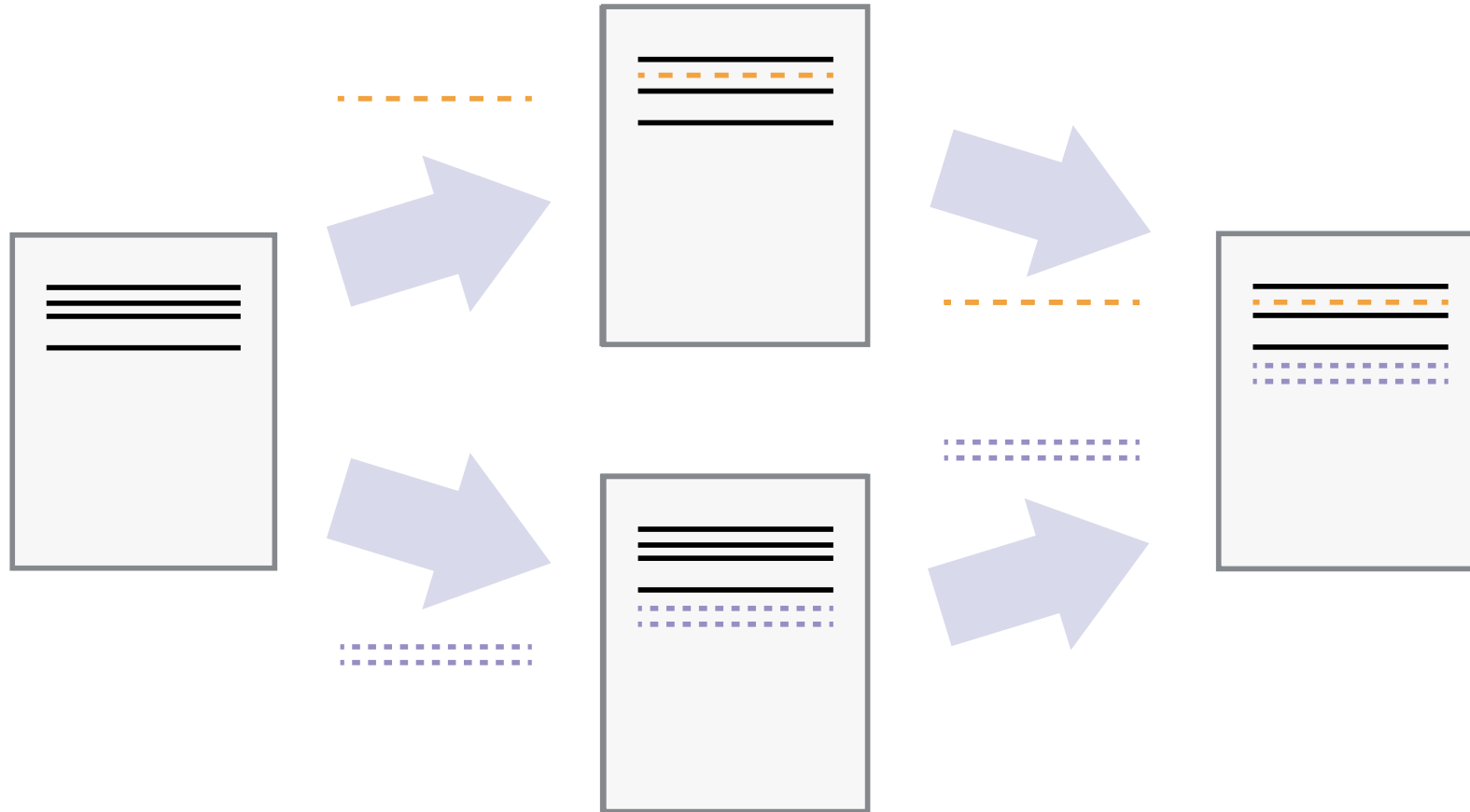
- Who can relate with the following development of files?
 - Assignment.docx
 - Assignment 2.docx
 - Assignment 2 – Copy.docx
 - Assignment Final.docx
 - Assignment Final Final.docx
 - Assignment FINAL.docx
- Who has used a collaborative document (e.g. Google docs, Colab)?



Changes



Versions and collaboration



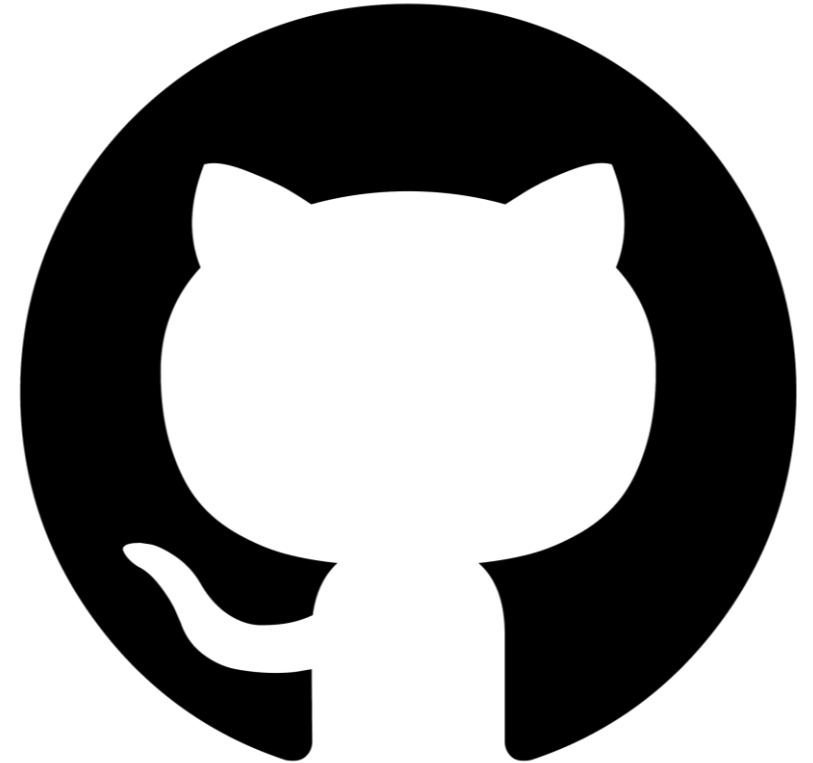
What is Git?

- Git is the **most used version control system**.
 - More than 90% of software projects around the world use Git.
- It is **free, open source, fast and scalable**.
- It was created in 2005 by Linus Torvalds
 - The same creator of Linux OS.
- It keeps track of the entire **history** of a project and allows **collaboration** of any size.
 - Git + GitHub(Gitlab)



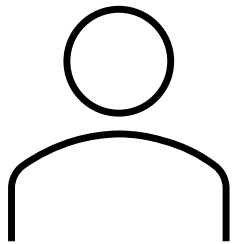
What is GitHub?

- Developer **platform**
<https://github.com>
- It allows developers to
 - create,
 - store,
 - manage and
 - share their code.
- Here, we store our project histories.
- Other alternatives are [Bitbucket](#) or [GitLab](#)

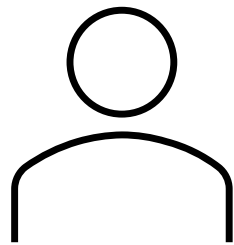


Today's agenda

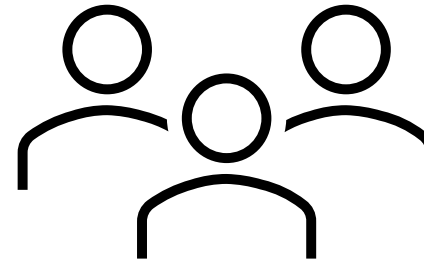
1. Local set up
and basics



2. Basics and
remote set up



3. Collaboration



Celebration!



“Version control might be the most important topic we teach, but Git is definitely the most complicated tool”

Jargon alert!

commit pull
config HEAD
repository init fetch cherry-pick merge
conflict ammend hash push branch
git reset stash clone GitHub
ssh add checkout show log revert
diff remote origin fork

Jargon alert!

Set up

init
config

Tracking changes

add
commit

git branch
hash repository

Block 1

History

log
diff
show

conflict
HEAD

Block 2

Remote repository

remote
clone

fetch
pull
push

GitHub ssh
origin

Block 3

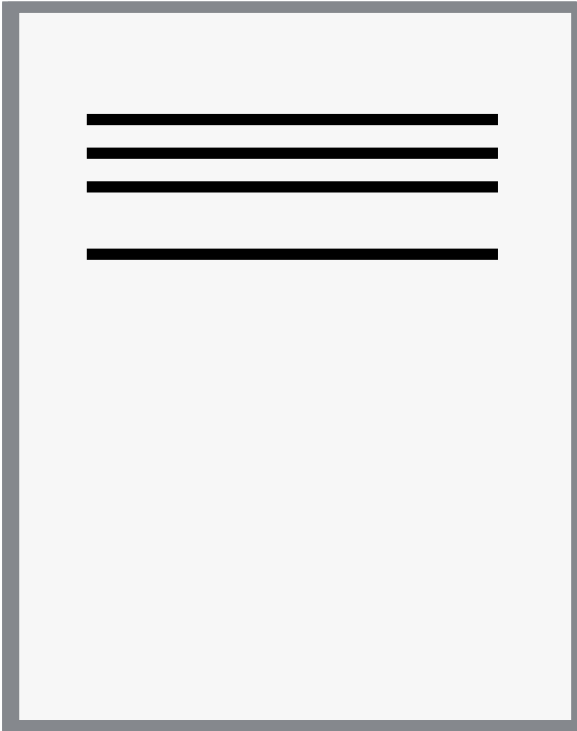
revert
ammend
cherry-pick
stash
reset
fork
...
checkout
merge

Basics



Working directory

Commits



Snapshot of the folder

Author

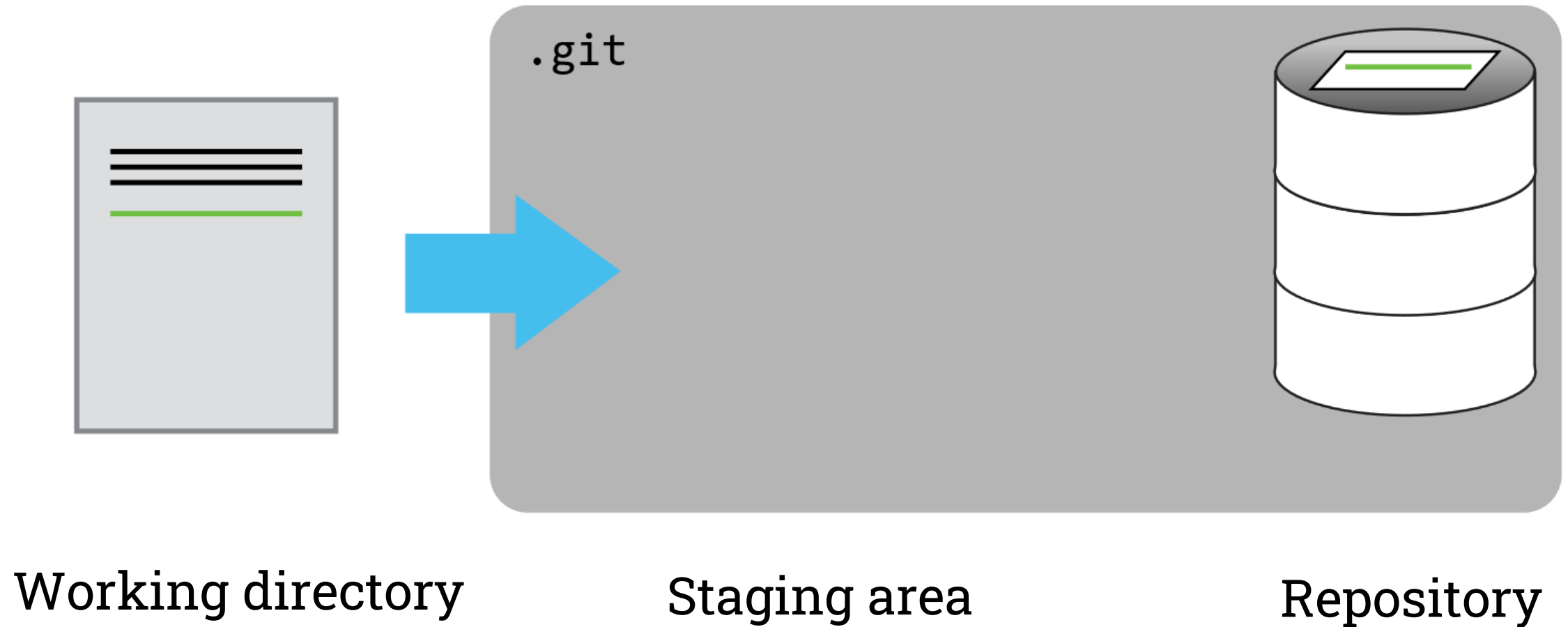
Email

Date

Message (an explanation)

Let's Git!

Tracking Changes



Exercise 1

1. Create a new text file.
2. Write some additional lines to your file.
3. Add the changes to the staging area.
4. Commit them.

Exercise 2

1. Delete some lines from your file.
2. Add the changes to the staging area.
3. Commit them.

Exercise 3

1. Create a new file and add some text.
2. Add the changes to the staging area.
3. Commit them.

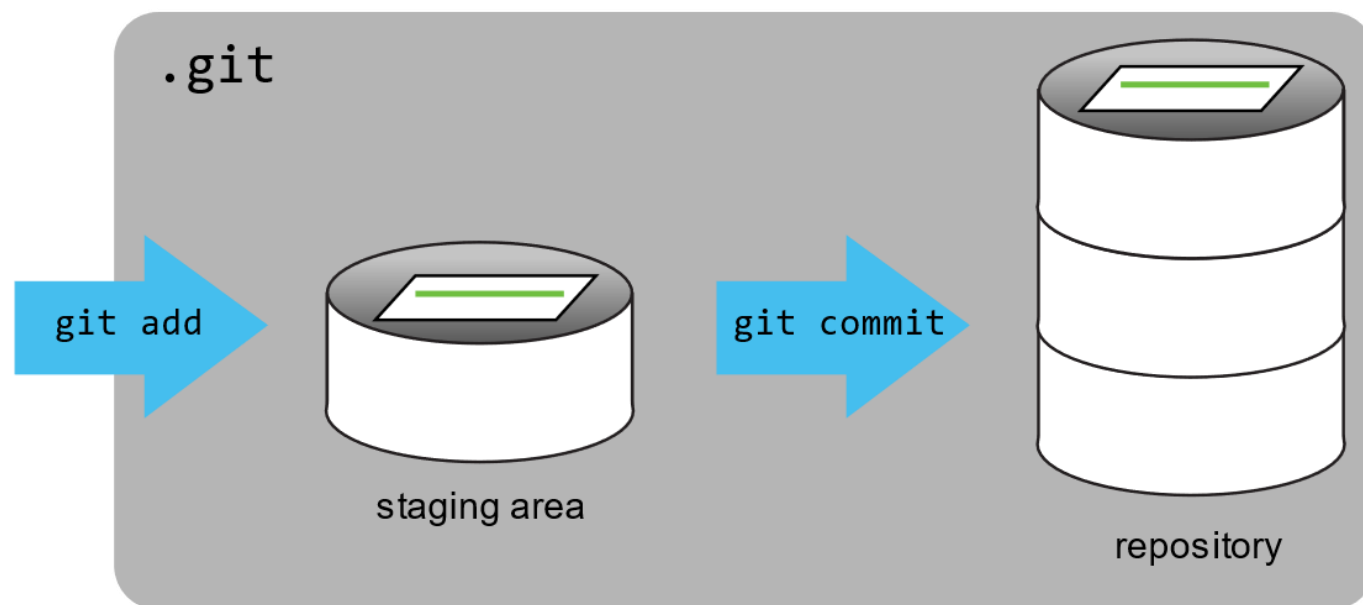
Notes

- Run git status after every step.
- Run git log after step 3.
- Repeat the exercise a couple of times

Break

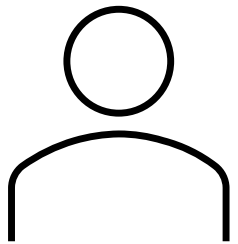
Key takeaways

- Git uses the pattern
 - `git <verb> --options`
- For setting up your name and email
 - `git config --global user.name "your name"`
 - `git config --global user.email your@email.com`
- To initialize a repository, we use `git init`
- We can commit our changes with
 - `git add`
 - `git commit`
- We can see our changes with
 - `git diff`
 - `git log`

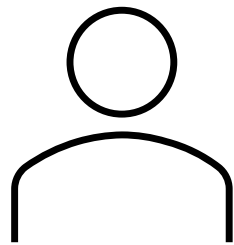


Today's agenda

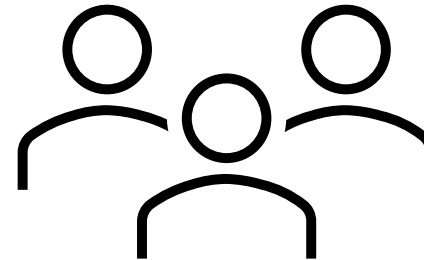
1. Local set up
and basics



2. Basics and
remote set up



3. Collaboration



Celebration!



Part 2

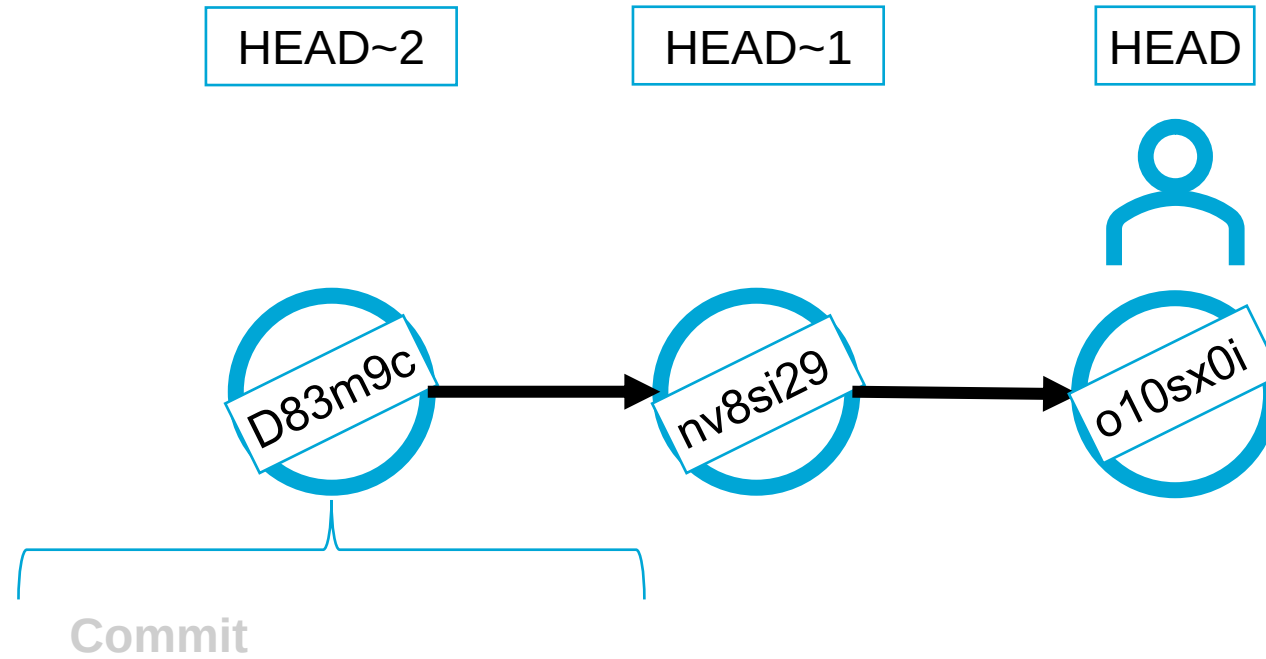
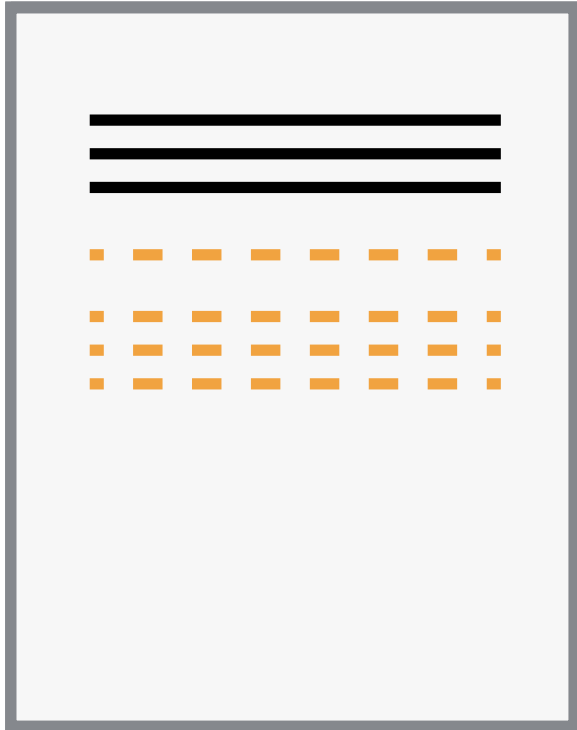
Basics II

- How to see the difference with previous versions
- How to show the changes introduced in a commit
- How to ignore files

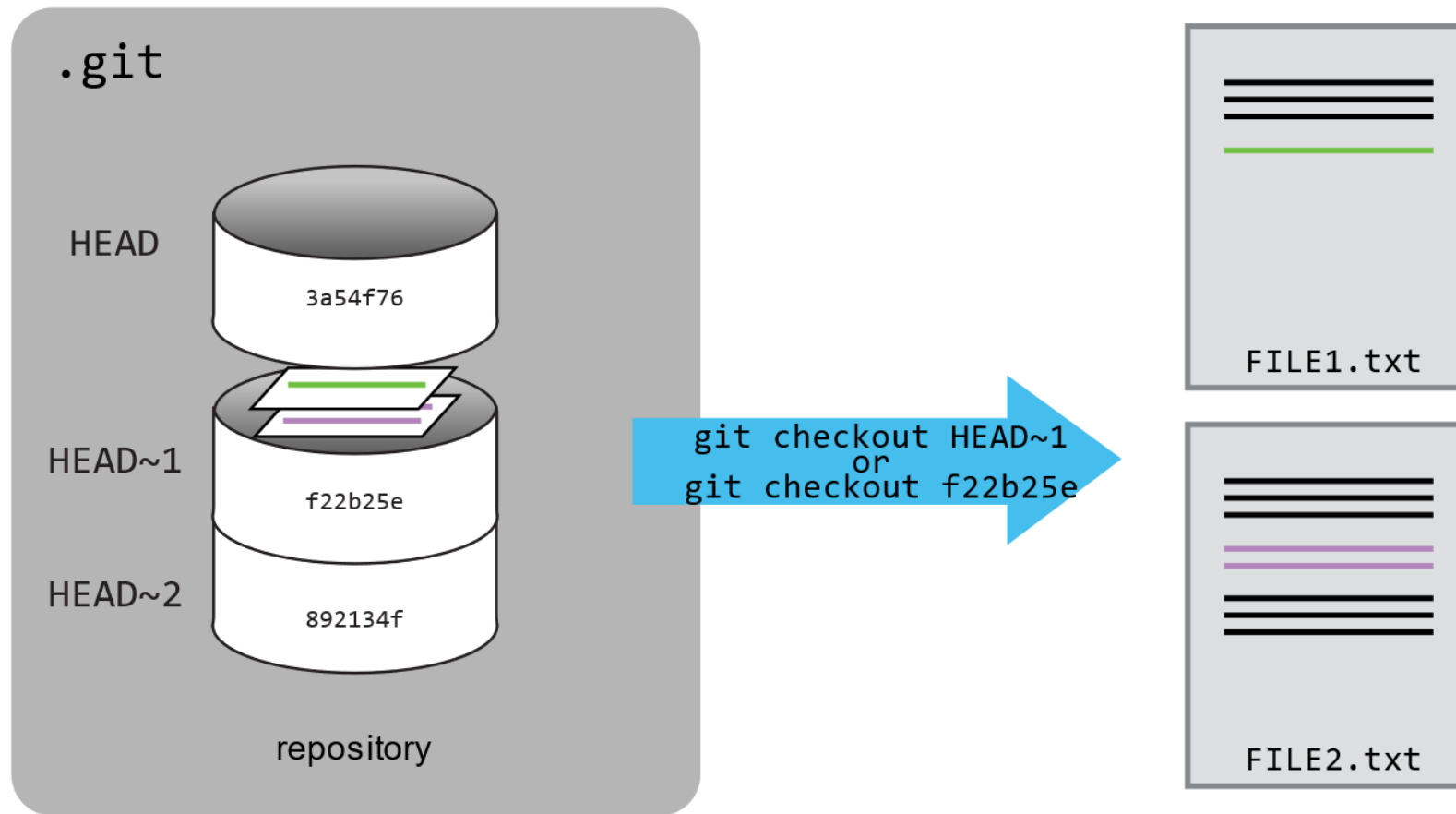
Remote set up

- Guide to GitHub
- How to connect your local computer to the remote storage of GitHub
 - How to set up your SSH keys

Time travel



git checkout



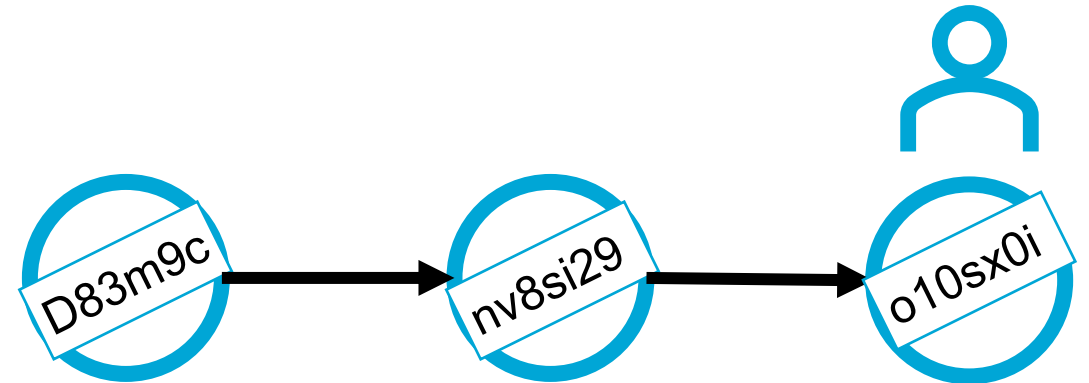
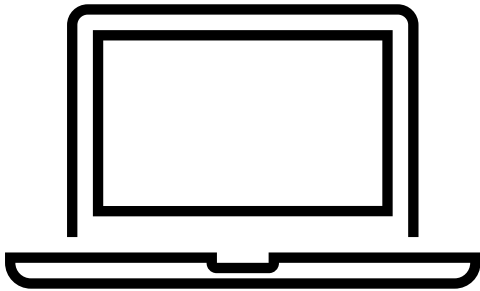
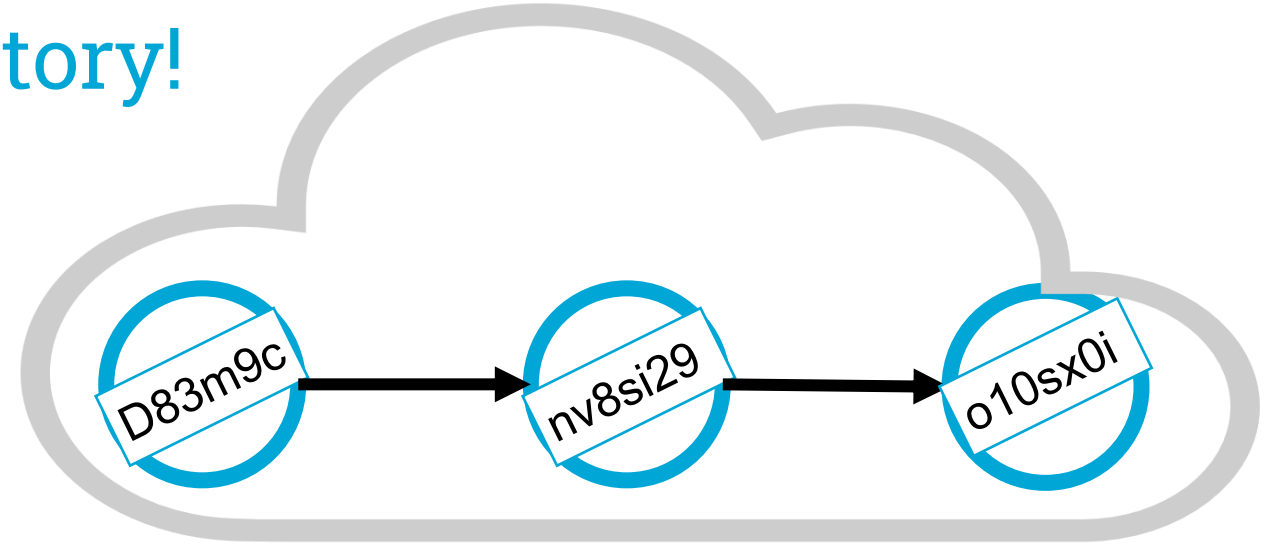
Exercise 4 – The alphabet

1. Create a new text file called alphabet.txt
2. Modify the file by adding the letter “A”.
3. Add and commit those changes.
4. Modify the file by adding the letter “B”.
5. Add and commit those changes.
6. Modify the file by adding the letter “C”.
7. Add and commit those changes.
8. Recover the versión of the file that only had “A” in it.

Key takeaways

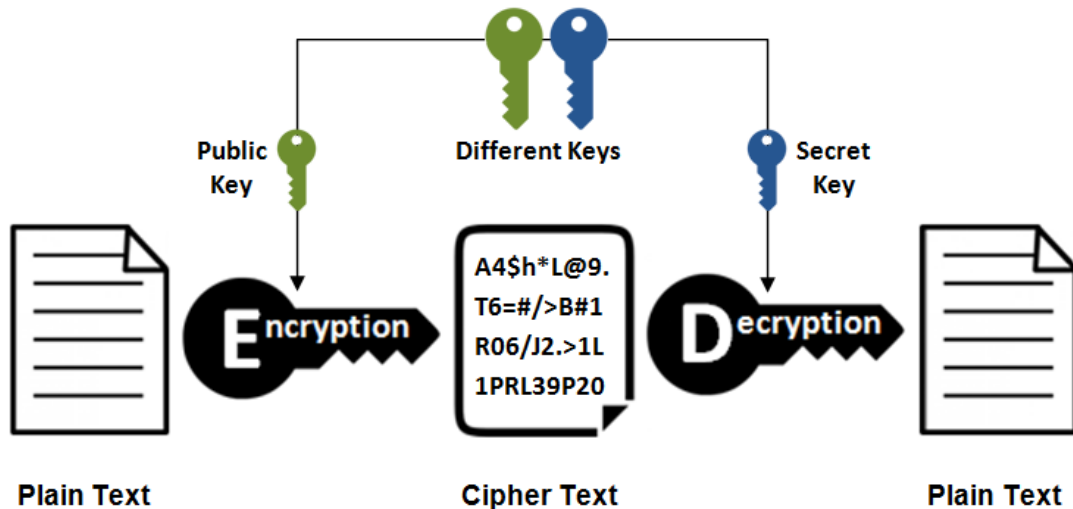
- git diff displays differences between commits.
- git checkout recovers old versions of files.
- The .gitignore file tells Git what files to ignore.

Let's create a remote repository!



SSH Keys

Asymmetric Encryption



- Open Git Bash.
- `ssh-keygen -t ed25519 -C "your_email@example.com"`
 - > Enter passphrase (empty for no passphrase): [Type a passphrase]
 - > Enter same passphrase again: [Type passphrase again]
- `eval "$(ssh-agent -s)"`
- `ssh-add ~/.ssh/id_ed25519`

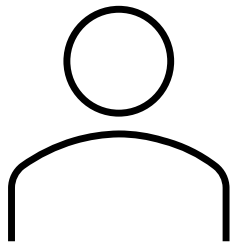
Best way, search for the documentation:

[Generating a new SSH key and adding it to the ssh-agent - GitHub Docs](#)

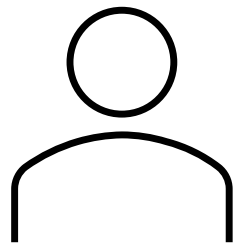
Break

Today's agenda

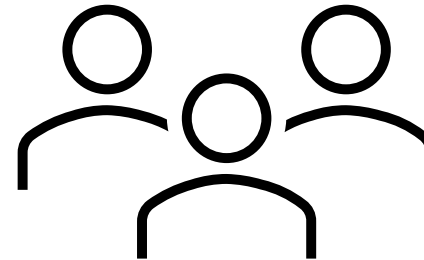
1. Local set up
and basics



2. Basics and
remote set up



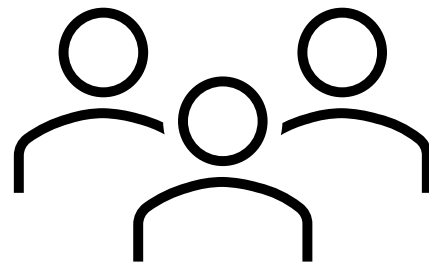
3. Collaboration

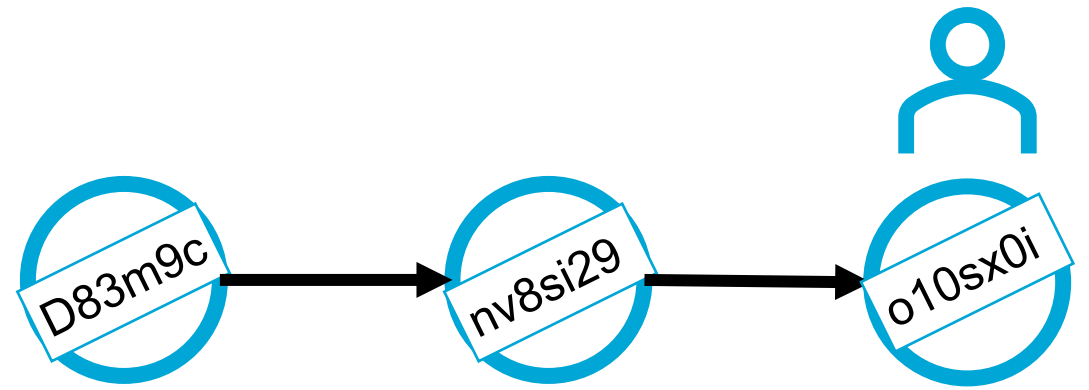
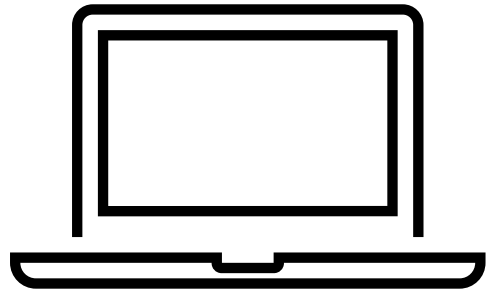
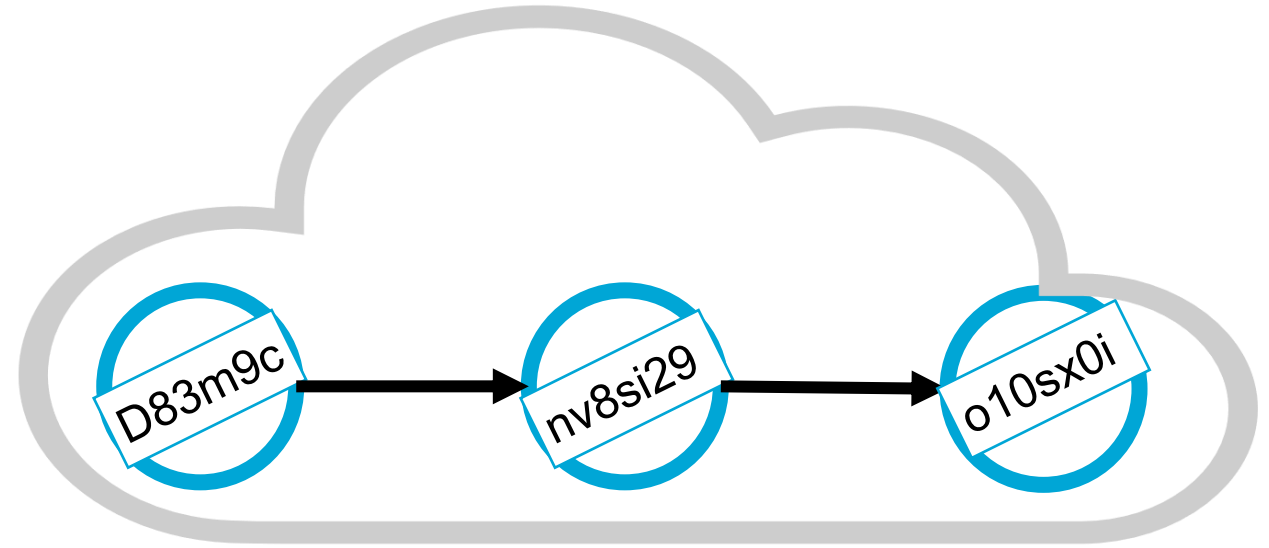


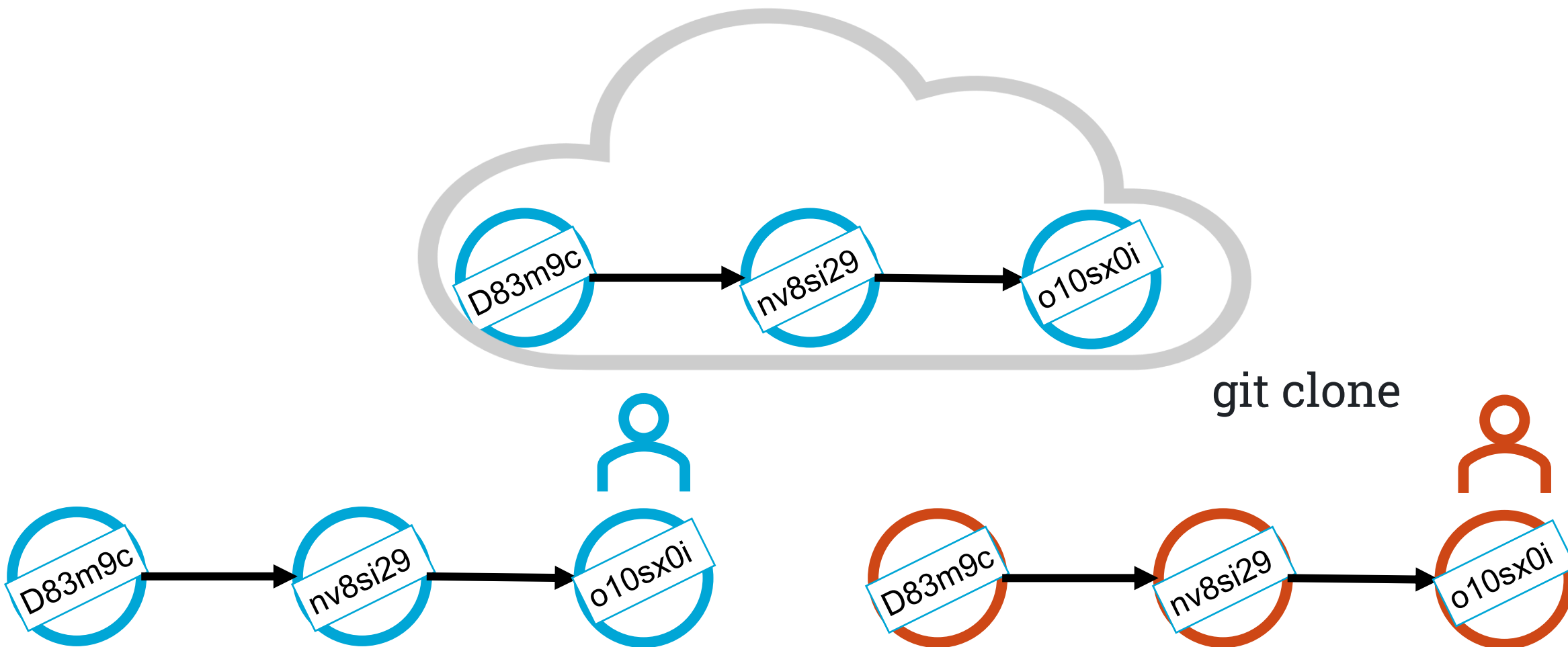
Celebration!



Let's learn about
GitHub!







Collaboration

Exercise 5



1. Add your colleague(s) as collaborator(s)
2. Add some text lines in one of your local files.
3. Add and commit your changes
4. Push your changes
5. Fetch the changes



1. Clone the repository to your local computer
2. Make a new file and add some text lines.
3. Add and commit your changes
4. Push your changes
5. Fetch the changes

6. Pull the changes

Key takeaways

- A local Git repository can be connected to one or more remote repositories.
- Use the SSH protocol to connect to remote repositories.
- `git push` copies changes from a local repository to a remote repository.
- `git pull` copies changes from a remote repository to a local repository.
- `git clone` copies a remote repository to create a local repository with a remote called `origin` automatically set up.

Celebration time!

